

CyphaLM: Explicit Compression, Uncertainty, and Self-Organisation for Online Language Modelling

Author: Odin Loch **Status:** Draft — rewritten 2026-07-12; **narrative reconciled 2026-07-17** against canonical D17 hybrid 300k BPC pin **2.873** (b0d39e7, docs/reports/BASELINE_PIN_CANONICAL_2026-07-17.md). Describes the current **native C++** CyphaLM (hybrid GRIA+LSTM, trained by real backpropagation). The Python prototype this paper originally described (cypha_lm/) was fully decommissioned in Phase P7 (see CHANGELOG.md, “[Unreleased] > Removed”: *“Python runtime decommissioned (P7): ... cypha_lm/ ... removed from the product path. Native C++ ... is the sole runtime.”*). Metrics below are sourced from bench/BASELINE_LOCK.json, bench/BASELINE_REPORT.md, and docs/RESEARCH_STATUS.md; every figure/table is annotated with its source, and any claim that could not be re-verified against the native implementation is explicitly flagged rather than carried over from the old draft. **Canonical rule:** cite **2.873** for the locked baseline, production gate, and RPSM stop/go target; other 300k hybrid figures (2.892, 2.864, etc.) are historical run artifacts only.

Abstract

Conventional neural language models entangle lossy distribution compression, lossless residual encoding, statistical density estimation, and self-organised feature discovery in a single optimisation process. **CyphaLM** keeps each mechanism as a separate, named subsystem — **Izaak** GF(2ⁿ) fixed-embedding lookup (zero trainable embedding parameters, structurally collision-free by construction), **CellAI** multi-scale state-space temporal layer (O(1) recurrent state update per token), **CyphaDIF** Normal-Inverse-Gamma (NIG) expert field (explicit epistemic/aleatoric uncertainty decomposition, optionally kernel-LLR-blended routing), and a **GRIA** low-rank alpha-projection output head — but, unlike the Python prototype this paper originally described, the current native C++ implementation (native/src/cyphalm/) trains all of these end-to-end with **real gradient descent / backpropagation** (truncated BPTT on the char-LSTM head, cross-entropy gradient steps on GRIA’s low-rank factors and per-token α , and a windowed BPTT update on the SSM projection), not a gradient-free NumPy scheme. GRIA’s output is blended, at the logit level via a learned sigmoid weight, with a parallel trainable character-level LSTM head; the model is referred to internally as **“hybrid GRIA+LSTM.”** On the D17 benchmark (WikiText-2, official train/valid split, 300k training characters), this hybrid model achieves **2.873 bits-per-character (BPC)** held out, beating a bigram baseline (**3.478**, $\Delta -0.61$) and a trigram baseline (**4.398**, $\Delta -1.53$); a plain (non-hybrid) char-LSTM baseline trained under the same harness reaches 2.979 BPC, i.e. the hybrid’s advantage over n-gram baselines is real but the model’s own learned blend weight assigns roughly **99.6% of the logit mix to the LSTM branch**, so at the current operating point the GRIA/SSM/DIF pathway is contributing comparatively little of the headline win. Online domain adaptation reduces held-out BPC by roughly **0.25–0.29** on out-of-distribution WikiText text. Single-threaded production training throughput is **252.2 characters/second** (MSVC, 2026-07-12 profiling pass). An intelligence-profile instrumentation hook computes a composite criticality score $\kappa = 1 - (1/7) \cdot \sum |P_i - P^*_i|$ over seven statistics (α , D_{eff} , σ_{branch} , τ , r_{eu} , Lipschitz, calibration); at 5,000-step ablation scale κ rises from 0.826 (no auxiliary loss) to 0.870 with the `--math-integration` profile-guided navigation loss enabled, still short of the internal target $\kappa \approx 0.89$. Several of the old draft’s headline numbers (Izaak kNN accuracy, expert purity, toy-vocabulary perplexity, transformer parameter-efficiency comparison) were specific to the now-removed Python prototype and could not be reproduced against the native implementation within this rewrite’s scope; they are reported below as historical, superseded values, not current results.

Key results (native, 2026-07-12):

Experiment	Metric	Value	Source
D17 (WikiText-2, 300k train)	Hybrid GRIA+LSTM held-out BPC (pinned)	2.873	bench/BASELINE_LOCK.json:d17_hybr
D17 (WikiText-2, 300k train)	Native build6 sweep BPC (historical)	2.892	docs/native/CYPHALM_NATIVE_BENCH_

Experiment	Metric	Value	Source
D17 (WikiText-2, 300k train)	Overnight native run BPC (historical sweep, not lock)	2.864	bench/BASELINE_LOCK.json:overnigh
D17 baselines @ 300k	Bigram / Trigram / char-LSTM-only	3.478 / 4.398 / 2.979	docs/RESEARCH_STATUS.md "Language model (D04 + D17)"
D17 ablation	GRIA-only stack (no LSTM) BPC	3.838	docs/RESEARCH_STATUS.md
D17D online adaptation	Δ BPC on OOD WikiText text	≈ -0.25 to -0.29 (two runs cited)	docs/RESEARCH_STATUS.md (Confirmed-properties table: -0.250 ; post-upgrade table: -0.288)
D21 RPSM (alternative sequence layer)	Held-out BPC @ 300k	7.336 (far behind hybrid)	bench/BASELINE_LOCK.json:rpsm_res
Math-integration ablation @ 5k steps	BPC / κ , baseline $\rightarrow$ with --math-integration	4.001/0.826 $\rightarrow$ 3.983/0.870	bench/BASELINE_LOCK.json:math_int
Training throughput	Single-threaded chars/sec (production D17 config)	252.2	docs/reports/PERFORMANCE_PROFILE_
CellAI SSM diagnostic	λ_{fast} / λ_{slow} decay rates, no state collapse/explosion	0.3679 / 0.9512, checks passed	bench/BASELINE_REPORT.md D17 ssm_diagnose
Task routing / discovery (closest current native analogue to "expert purity")	Adjusted Rand Index	1.000	docs/RESEARCH_STATUS.md D16A (a classification task, not LM expert routing — see §3.3)
Shared-model continual learning	Forgetting score (NOT zero-forgetting)	0.813	docs/RESEARCH_STATUS.md D16B
Per-task-isolated continual learning	Forgetting score	0.000	docs/RESEARCH_STATUS.md D16F

Superseded (Python-prototype-only, not re-verified on native — see §3 and §5 for details): Izaak kNN accuracy 0.0125, GF-rank Spearman 0.1434, synthetic 2-topic expert purity 1.0000, toy-vocabulary perplexity 12.9032, code-corpus 0.6146 bits/char, NIG-vs-dropout ECE calibration, and the transformer parameter-efficiency comparison (Fig. 10). These numbers live only in `paper/figures/expON_*.json`, which are historical artifacts of the decommissioned Python stack (`cypha_lm/`); see the per-section notes below for what, if anything, the native implementation has as a current equivalent.

1. Introduction

1.1 Four mechanisms in neural networks

Lossy compression (GRIA $\alpha \rightarrow 1$), lossless residual channel (epistemic uncertainty), statistical $P(Y|X)$ approximation, and self-organisation at the edge of chaos ($\alpha \approx 0.5$). This framing is unchanged from the original draft; it motivates the subsystem split, independent of which runtime implements it.

1.2 Limitations of entangled mechanisms

Opacity, parameter inefficiency, no calibrated uncertainty, catastrophic forgetting. **Update:** the native implementation’s own bench results now directly confirm the forgetting risk for at least one configuration — D16B measures **81.3% forgetting** in a shared-model multi-task setting, refuting an earlier “zero forgetting” claim for that regime. Zero forgetting is confirmed only for per-task-isolated model files (D16F, D16E) — see §5.8 and §6.4.

1.3 CyphaLM thesis

Explicit, named mechanisms are more efficient and interpretable than an entangled end-to-end network — **and**, contrary to the original draft’s framing, explicit mechanisms are not incompatible with gradient-based training: the native CyphaLM trains every one of its four subsystems (SSM, DIF, GRIA, LSTM) via real backpropagation while keeping them as separately-inspectable modules (separately loggable κ/D_{eff} /BPC statistics via the intelligence-profiler hook, separate ablation switches in `cyphalm_config.hpp`, separate checkpoint sections). The “explicit” claim is about modularity and instrumentation, not about avoiding gradients.

2. Background

- 2.1 Compression-based learning (MDL, GRIA-NMP)
- 2.2 Normal-Inverse-Gamma posteriors (implemented natively in `cyphalm_nig_expert.hpp` / `nig_state_cell.hpp`)
- 2.3 State space models ($O(n)$ sequence advantage; native CellAI implementation gives $O(1)$ *per-token* recurrent update, see §3.2)
- 2.4 Grand Unified Law: $\alpha = 1 - H(f(X)) / H(X)$ (native alpha-spectrum instrumentation: `cyphalm_alpha_spectrum.hpp`, `ssm_diagnose.hpp`)

This background is conceptually unchanged; all four cited constructs still exist in the native codebase under the same names.

3. CyphaLM Architecture

The native pipeline (`native/include/cypha/cyphalm/cyphalm_model.hpp`) is: token $\rightarrow$ **Izaac** fixed embedding (feeds the SSM path) $\rightarrow$ **CellAI** SSM $\rightarrow$ **CyphaDIF** expert field $\rightarrow$ **GRIA** low-rank projection head, whose output logits are blended at the logit level (learned sigmoid weight `hybrid_blend_logit`) with a parallel, separately-embedded, trainable **char-LSTM** head. All trainable components are updated by gradient descent every character step (`CyphaLMModel::train_step`).

3.1 Izaac $GF(2^n)$ embeddings

Current status: exists, same name and construction, in `native/src/cyphalm/embed_table.cpp` (`EmbedTable`). Token IDs are mapped through a $GF(2^n)$ permutation polynomial (irreducible-polynomial multiplication + a coprime exponent, matching the construction `galois.GF(2**n)` used historically) into a fixed lookup table generated once at construction and never updated by gradient descent — it genuinely has **zero trainable parameters**, feeding the CellAI SSM input. (Note: the char-LSTM head has its own, separate, *trainable* embedding table `E`; only the Izaac/SSM path is parameter-free.)

Collision rate: because the underlying map is a bijection over $GF(2^n)$ (the exponent is chosen to be coprime with the multiplicative group order, guaranteeing an injective — hence collision-free — mapping for any set of token IDs smaller than the field size), a collision rate of **0.0 is a structural guarantee of the construction, not merely an empirical measurement** — this actually strengthens, not contradicts, the old draft’s “collision rate = 0.0” claim.

kNN accuracy 0.0125 / GF-rank Spearman 0.1434: these are specific to the old Python prototype’s evaluation harness (nearest-neighbour semantic-proximity probe on a 512-token vocabulary; see `paper/figures/exp01_embedding_metrics.json`) and have **not been re-measured against the native implementation**. Native Izaac is not designed to preserve semantic proximity between tokens — it is an explicit, structured, zero-parameter substitute for a learned embedding, not a semantic one — so this specific metric may not even be the right one to re-measure; we flag it as **not re-verified / possibly not meaningful for the current design goal**, rather than repeating the old number as current.

(Historical figure: `paper/figures/exp01_embedding_metrics.json` — Python-prototype-only; no native re-measurement exists yet.)

3.2 CellAI SSM temporal layer

Current status: exists, same name, in `native/src/cyphalm/cellai_ssm.cpp` (CellAISSM). A multi-scale rank-2 SSM with separate fast/slow tracks per layer, optional spectral-PDE circulant state transition, and optional sparse Hebbian/temporal-SOM extensions. Each token step is a single fixed-size recurrent state update (**O(1) per token**, matching the original claim), not a full-sequence O(n) reprocessing pass.

Current native measurement (D17 `ssm_diagnose` probe, `bench/BASELINE_REPORT.md`): fast/slow decay rates $\lambda_{\text{fast}} = 0.3679$, $\lambda_{\text{slow}} = 0.9512$; state-norm checks pass (`checks_passed: true`, no collapse or explosion in fast/slow/context tracks over a 128-step probe); GRIA field-projection is connected to routing (`projection.connected_to_routing: true`). This is a real, current, native measurement — unlike the old draft’s N/A placeholder for this row.

(Historical figure: `paper/figures/exp02_ssm_metrics.json` compares a Python-prototype CellAI against an RNN/transformer on tiny synthetic sequences up to length 100 — not reproduced natively; no native RNN/transformer comparison exists in this repo.)

3.3 CyphaDIF expert field

Current status: exists, same name, in `native/src/cyphalm/cyphalm_dif.cpp` (CyphaDIF) + `cyphalm_nig_expert.hpp`. Routes the SSM field through NIG experts, producing an explicit epistemic/aleatoric variance decomposition per prediction; optionally blends its linear-LLR routing softmax with a KernelMemory Nyström (or RFF) kernel score (`use_kernel_llr`, shipped as “H04”).

Expert purity 1.00 (old claim): this number came from a synthetic two-topic Python routing benchmark that has no direct analogue in the current native LM bench domains. **We could not re-verify it as stated.** The closest current native measurement of routing/discrimination quality is **D16A task discovery, Adjusted Rand Index = 1.000** (`docs/RESEARCH_STATUS.md`) — but that is a *classification* continual-learning benchmark, not the LM character-routing task the old “purity” number described, so it is cited here as supporting evidence that CyphaDIF’s routing *can* be perfect on a well-separated task, not as a direct replacement metric. Within the D17/D04 language-modelling domains specifically, the alpha-spectrum probe (§3.4) currently reports `n_experts = 1` — i.e. only one expert is active in the production hybrid config. D17B diagnostics (`docs/reports/D17B_EXPERT_REPORTING_2026-07-12.md`, shipped e6d95d2) established this is **genuine novelty-gated dynamics** on the default cold-start profile (not a warm-start reporting bug): field inputs rarely trigger a second expert, so a single dominant expert is the expected operating point at current mean $\alpha \approx 0.095$.

(Historical figure: `paper/figures/exp03_experts_metrics.json` — Python-prototype synthetic 2-topic corpus; not reproduced natively.)

3.4 GRIA alpha-projection

Current status: exists, same name, in `native/src/cyphalm/gria_lowrank.cpp` (GRIALowRank) — and, unlike the old draft’s framing, it is trained by real gradient descent. Logits are $\alpha_k * z_k + (1 - \alpha_k) * \text{bias}_k$, where $z = V^T @ (U^T @ v)$ is a rank-32 low-rank factorization and α is a per-vocabulary-token, independently learnable parameter (`alpha_learnable = true`) updated every step

via `GRIALowRank::train_step`'s cross-entropy gradient (`cross_entropy_gradients` → `update_weights` / `update_alpha` / `update_bias`), not an online non-gradient rule.

Current native alpha-spectrum measurement: `docs/RESEARCH_STATUS.md`'s post-upgrade note reports **mean $\alpha \approx 0.095$** for the production hybrid model — “still low alpha; 1 active expert” — versus an earlier (2026-06-13, pre-upgrade) `bench/BASELINE_REPORT.md` D17B snapshot of mean $\alpha = 0.4981$, fraction-edge-of-chaos = 0.9961 (`n_experts` = 1 in both cases). We report both, dated, rather than picking one: the model's α spectrum has shifted over successive training-recipe changes and the most current (post-upgrade) reading is the low one. Combined with §3.3's `n_experts=1` and the abstract's “99.6% LSTM blend weight” finding, the honest current picture is that **GRIA's own compression/expert-richness signal is currently under-expressed in the production hybrid configuration** relative to the char-LSTM branch it is blended with.

(Historical figure: `paper/figures/exp04_alpha_metrics.json` — Python-prototype edge-of-chaos histogram at synthetic checkpoints; not reproduced natively at the same checkpoints.)

3.5 End-to-end pipeline

Token → Izaac (fixed, feeds SSM) → CellAI SSM → CyphaDIF (routing + NIG uncertainty) → GRIA low-rank α -head, **blended at the logit level with a parallel trainable char-LSTM head via a learned sigmoid weight** → $P(\text{next token})$. Trained end-to-end by gradient descent: char-LSTM via truncated BPTT-1 (`CharLSTMHead::backward`), GRIA via cross-entropy gradient steps, SSM projection via a windowed BPTT update (`CyphaLMModel::bptt_ssm_update`). This is the single largest architectural change versus the original draft: the pipeline shape is recognisable, but the training mechanism is not gradient-free.

4. Theoretical Properties

- 4.1 Epistemic variance bounds lossless residual mass
- 4.2 Per-token inference is $O(1)$ in the recurrent state size (not full-sequence $O(n)$ reprocessing) — confirmed for the native CellAI SSM (§3.2); training remains sequential (each step depends on the previous hidden state) and is not GPU-parallelisable without an algorithmic change (`docs/reports/PERFORMANCE_PROFILE_2026-07-12.md` §5).
- 4.3 Bounded expert creation — native `max_experts` cap in CyphaDIF; current production LM configuration reports a single active expert (§3.3), so this bound is not being stress-tested at its current operating point.
- 4.4 $\alpha \approx 0.5$ maximises bits-per-parameter — the current production hybrid model's measured mean α (≈ 0.095 , §3.4) is well below this target, an open gap rather than a confirmed property.

5. Experiments

All experiments below are re-grounded in current native bench results (`bench/BASELINE_LOCK.json`, `bench/BASELINE_REPORT.md`, `docs/RESEARCH_STATUS.md`) as of 2026-07-12. Where no native equivalent experiment exists, this is stated explicitly rather than silently keeping the old number.

5.1 Embedding benchmark (historical Fig. 1)

See §3.1. **Not re-measured natively.** Old Python-prototype values (kNN accuracy 0.0125 for Izaac vs. 0.0025 random / 0.0100 frequency-init learned, collision rate 0.0 for all three) are historical only; the collision-rate-zero property is, however, independently re-derivable as a structural guarantee of the current native $GF(2^n)$ construction (§3.1).

5.2 SSM sequence capacity (historical Fig. 2)

Not re-measured natively as an RNN/transformer-vs-CellAI lag-accuracy sweep. The current native equivalent evidence is the `ssm_diagnose` probe (§3.2): fast/slow decay rates well-separated (0.3679 / 0.9512), no state collapse/explosion over a 128-step probe on the production D17 model.

5.3 Expert self-organisation (historical Fig. 3)

Not re-measured natively on an equivalent synthetic two-topic corpus. See §3.3 for the closest current native evidence (D16A routing ARI = 1.000, a different task) and the confirmed `n_experts=1` genuine-dynamics finding on D17B.

5.4 Alpha spectrum emergence (historical Fig. 4)

Current native measurement: mean $\alpha \approx 0.095$ (post-upgrade production config) vs. 0.4981 (2026-06-13 pre-upgrade snapshot); see §3.4 for full discussion. This is now a current, cited, native number — the old draft’s N/A placeholder is resolved, but the resolved value shows a gap against the $\alpha \approx 0.5$ theoretical target (§4.4), not a confirmation of it.

5.5 Language modelling (D04 Gutenberg / D17 WikiText-2)

Replaces the old draft’s “toy vocabulary” and “code corpus” Python-prototype experiments (`paper/figures/exp05_toy_lm_metrics.json`, `exp06_code_lm_metrics.json`), which have no equivalent in the current native bench domains and are not carried forward as current results.

D17 WikiText-2, held-out BPC by training scale and context mode (`docs/RESEARCH_STATUS.md` “Language model (D04 + D17)”):

Train scale	Context mode	CyphaLM	Bigram	Trigram	Char-LSTM baseline
40k	<code>gria_ngram</code>	4.154	3.914	4.398	—
300k	<code>hybrid_gria_lstm</code> (production)	2.873	3.478	4.398	2.979
300k	<code>gria_ngram</code> (GRIA-only ablation)	3.838	3.478	4.398	2.979

D04 Gutenberg (Moby Dick fallback) shows the same qualitative pattern: 300k hybrid BPC 2.993 vs. bigram 3.633, trigram 3.424 (trigram beats bigram here; hybrid still ahead of both). **The production hybrid model beats bigram and trigram baselines and a dedicated char-LSTM baseline** on WikiText-2 at 300k training characters — a materially different and better result than the old draft’s headline “4.50 BPC, above bigram” claim, but see the abstract’s caveat that the LSTM branch, not GRIA/SSM/DIF, carries most of this win.

5.6 Uncertainty calibration (historical Fig. 7)

Not re-measured natively as an ECE-vs-dropout-baseline comparison (`paper/figures/exp07_calibration_metrics.json` is Python-prototype-only). The closest current native uncertainty-quality evidence is out-of-distribution detection AUROC on the general CyphaDIF classification bench (not the LM specifically): cross-domain mean OOD AUROC **0.844** (`docs/RESEARCH_STATUS.md`, “Confirmed architectural properties”), D12A intrusion-detection OOD AUROC **0.889**. A dedicated LM-calibration (ECE) re-measurement on the native hybrid model has not been performed.

5.7 Online adaptation

Current native measurement (D17D): held-out BPC improves by roughly **0.25–0.29** after online adaptation on out-of-distribution WikiText text — docs/RESEARCH_STATUS.md cites both **−0.250** (summary “Confirmed architectural properties” table) and **−0.288** (detailed post-upgrade, 40k WikiText table) from what appear to be two different runs/configurations; we report the range rather than picking one arbitrarily. This replaces the old draft’s Python-prototype ppl_online/ppl_frozen perplexity-improvement figure (paper/figures/exp08_adaptation_metrics.json, a synthetic-corpus number ~2248 perplexity-points, not a meaningful comparison to the current BPC-based measurement).

5.8 Catastrophic forgetting

Materially revised from the old draft, which had no native data (N/A placeholder) and no reason to expect the outcome below. Current native measurement (docs/RESEARCH_STATUS.md, D16 domain):

Configuration	Forgetting score	Verdict
Shared model, block training on task B	0.813	❑ Substantial forgetting — not zero-forgetting
Per-task isolated model files (save/reload)	0.000	❑ Zero forgetting, but only by architectural isolation
Save/restore fidelity (single task)	retention ratio 1.000	❑ Lossless save/restore

This directly contradicts any implicit “no catastrophic forgetting” framing in the original draft’s §1.2/§4. The corrected claim, per the maintainers’ own prescribed language: *“no forgetting per isolated model file; shared-model continual learning is an open problem”* (docs/RESEARCH_STATUS.md, Priority 5).

5.9 Parameter efficiency

Not re-measured natively. The old draft’s Fig. 10 comparison (paper/figures/exp10_efficiency_metrics.json) pits a Python-prototype CyphaLM (10.05 BPC at ~70k–220k params) against a transformer baseline that reports implausibly near-zero BPC (~0.0002) on what is evidently a tiny memorised/overfit synthetic task — this comparison was not meaningful even at the time and is not carried forward. No fair, parameter-matched CyphaLM-vs-transformer comparison exists for the current native hybrid model. The currently measured efficiency figure for the native implementation is **training throughput**, not a parameter/BPC efficiency curve: **252.2 characters/second single-threaded** on the production D17 configuration (MSVC, windows-vs2026-release), up from an initial ~96–126 chars/sec baseline via allocator-churn fixes and cache-friendly matrix-transpose loop reordering, with the D17 BPC pin confirmed bit-identical before and after (docs/reports/PERFORMANCE_PROFILE_2026-07-12.md).

6. Discussion

- 6.1 Epistemic residual as anti-hallucination signal — unchanged in spirit; native CyphaDIF still exports epistemic/aleatoric variance per prediction (DIFPredictOutput), and REST /generate exposes an epistemic-halt gate on r_eu.
- 6.2 Comparison to SSM literature (Mamba, S4) — unchanged; CellAI remains a rank-2 multi-scale SSM, not a selective-state-space (Mamba-style) design, though a SelectiveSSM class now exists natively as a separate, not-yet-default, alternative (native/include/cypha/cyphalm/selective_ssm.hpp).
- 6.3 Comparison to MoE literature — softened per §3.3/§3.4: the production LM configuration currently reports a single active expert (genuine D17B dynamics, not a reporting bug), so a meaningful MoE-style comparison (multiple specialised experts) is not yet demonstrated for language modelling specifically at the current α operating point.

- 6.4 Limitations — **materially expanded** versus the old draft:
 - Shared-model continual learning exhibits substantial (81.3%) forgetting; zero-forgetting holds only per isolated model file (§5.8).
 - The hybrid model’s headline BPC win is driven predominantly by its char-LSTM branch (~99.6% of the learned logit blend weight), not by the GRIA/SSM/DIF pathway the paper’s title and abstract foreground (§3.4, abstract).
 - GRIA’s mean α (≈ 0.095) is well below the $\alpha \approx 0.5$ “edge of chaos” target this architecture is theoretically motivated by (§4.4).
 - An alternative sequence layer (RPSM, “Option B”) trained on the same D17 task reaches only 7.336 BPC at 300k — far behind the hybrid model — attributed by the maintainers to RPSM lacking the hybrid’s BPTT depth rather than a training-loop bug (`docs/RESEARCH_STATUS.md`).
 - The `--math-integration` intelligence-profile-guided navigation loss measurably raises κ ($0.826 \rightarrow 0.870$ at 5k steps) but at the cost of BPC in at least one uncommitted 300k run (2.873 pinned hybrid vs. 3.073 BPC / $\kappa 0.860$ with math-integration, per `docs/reports/DEV_PLAN_2026-07-11.md` — not yet merged into the baseline lock) — i.e. there is an observed κ /BPC trade-off at production scale, not a free improvement on both axes.
-

7. Conclusion

CyphaLM’s native C++ hybrid GRIA+LSTM implementation makes the lossless residual (via CyphaDIF’s NIG epistemic/aleatoric split) and the compression parameter (via GRIA’s learnable per-token α) first-class, measurable, and now also **gradient-trained** quantities — a change from the original Python prototype’s gradient-free design, not merely a reimplementing of it. On its own headline task (D17, WikiText-2, 300k characters) it beats bigram and trigram baselines by a wide margin (2.873 vs. 3.478 / 4.398 BPC) and a dedicated char-LSTM baseline by a smaller one (2.873 vs. 2.979). The most important open finding from this rewrite is that the model’s own learned blend weight currently blends GRIA out almost entirely (~99.6% LSTM) in favour of the LSTM branch — so the “explicit mechanisms” thesis this paper argues for is not yet reflected in how much the winning configuration actually relies on GRIA, CellAI, or CyphaDIF individually. Closing that gap (raising GRIA’s contribution without losing BPC, and growing multi-expert routing beyond the current D17B single-expert operating point) is a more concrete, better-specified research target than the original draft’s now-superseded numbers.

References

1. Gu, A. & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752.
2. Amini, A., Schwarting, W., Soleimany, A. & Rus, D. (2020). Deep Evidential Regression. *NeurIPS*.
3. Grünwald, P. D. (2007). *The Minimum Description Length Principle*. MIT Press.
4. Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G. & Dean, J. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *ICLR*.
5. Langton, C. G. (1990). Computation at the edge of chaos: Phase transitions and emergent computation. *Physica D*, 42(1–3), 12–37.
6. Poole, B., Lahiri, S., Raghu, M., Sohl-Dickstein, J. & Ganguli, S. (2016). Exponential expressivity in deep neural networks through transient chaos. *NeurIPS*.
7. Tishby, N., Pereira, F. C. & Bialek, W. (2000). The information bottleneck method. arXiv:physics/0004057.
8. Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
9. Merity, S., Xiong, C., Bradbury, J. & Socher, R. (2017). Pointer Sentinel Mixture Models (WikiText-2). *ICLR*.
10. Rahimi, A. & Recht, B. (2007). Random Features for Large-Scale Kernel Machines. *NeurIPS*.

11. Kirkpatrick, J. et al. (2017). Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13), 3521–3526.
12. Murphy, K. P. (2007). Conjugate Bayesian analysis of the Gaussian distribution. Technical note (Normal–Inverse–Gamma).
13. Gu, A., Goel, K. & Ré, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces. *ICLR* (S4).
14. Sennrich, R., Haddow, B. & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*.

*Rewritten 2026-07-12 against the native C++ runtime (post-P7); BPC narrative reconciled 2026-07-17 (b0d39e7, canonical pin **2.873**); bibliography + native figure PNGs closed 2026-07-18. Primary sources: `bench/BASELINE_LOCK.json`, `bench/BASELINE_REPORT.md`, `docs/RESEARCH_STATUS.md`, `docs/reports/BASELINE_PIN_CANONICAL_2026-07-17.md`, `docs/reports/PERFORMANCE_PROFILE_2026-07-12.md`, `docs/reports/DEV_PLAN_2026-07-11.md`, `CHANGELOG.md`, and `native/src/cyphalm//native/include/cypha/cyphalm/` source. Historical Python-prototype metrics remain under `paper/figures/exp*.json`; current native bar charts are `paper/figures/native_fig_*.json/.png` (render: `scripts/render_native_paper_figures.py`).*